

Mateus Molina

Ermöglichung des Hardware- und Software-in-the-Loop durch die Einbindung von BaSyx und MATLAB/Simulink

Projektarbeit

Institut für Messtechnik und Sensorik
Technische Universität Kaiserslautern

Betreuung

Dipl.-Ing. Paaranan Sivasothy
Prof. Dr.-Ing. Jörg Seewig

Juni 2022

Zusammenfassung

Fill this area with your abstract

Schlagwörter: Simulink, BaSyx, Digitaler Zwilling.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation	1
1.2	Zielstellung	1
1.3	Gliederung der Arbeit	2
2	Theoretische Grundlagen	3
2.1	Der Simulationsprozess	3
2.1.1	Hardware-in-the-Loop (HiL)	3
2.1.2	Software-in-the-Loop	3
2.2	MATLAB/Simulink	4
2.2.1	Model-Based Systems Engineering (MBSE)	4
2.2.2	Simulink	4
2.2.3	MATLAB S-Functions	5
2.2.4	Personalisierung der Benutzeroberfläche eines Simulink-Blockes	5
2.3	Industrie 4.0	5
2.3.1	BaSys	5
2.3.2	Eclipse BaSyx	7
2.4	REST-Schnittstellen	9
3	Plannung und Durchführung	11
3.1	Entwicklungsprozess	11
3.1.1	Anforderungen	11
3.1.2	Konzept und Anforderungsdefinition	12
3.1.3	Funktionale Spezifikation	12
3.1.4	Systemarchitektur	13
3.2	Entwurf	15
3.2.1	Entwicklungsumgebung	15
3.2.2	Wrappen des BaSyx als S-Funktion	16
3.2.3	A1: Ablesen des Wertes aus einem SubmodelElement	18
3.2.4	A2: Aktualisierung des Wertes aus einem SubmodelElement	18
3.2.5	A3: Abrufen von AAS-Daten aus einem Registry Server	18
3.2.6	A4: Anzeigen von Simulink-konformen Fehlermeldungen	20
3.2.7	A5: Anwendung von mehreren Blöcken in demselben Simulink-Modell	23
3.2.8	A6: Betriebsmodus wechseln	23
3.2.9	A7: Personalisierung der Blockbenutzeroberfläche	23
4	Validierung	27
4.1	Beispielszenario	27
4.1.1	Setup vom BaSyx/Docker	27
4.1.2	Aufbau	27
4.2	Ausführung und Prüfen der Anforderungen	28

5 Zusammenfassung und Ausblick	31
5.1 Fazit	31
5.2 Ausblick	31
Abbildungsverzeichnis	33
Tabellenverzeichnis	35

Kapitel 1

Einleitung

1.1 Motivation

MATLAB/Simulink wird häufig in verschiedenen Industrie-Branchen von Ingenieuren großer und kleinerer Firmen eingesetzt, um Modelle mit wiederverwendbaren Komponenten und Bibliotheken zu simulieren [1] [2]. BaSys 4.0 ist ein vom BMBF gefördertes Forschungsprojekt, das von Fraunhofer IESE gemeinsam mit 14 weiteren Partnern aus dem Bereich der Produktionstechnik entwickelt wird, um digitale Zwillinge als digitale Repräsentanzen für die Produktion zu realisieren [3]. Eine Open-Source Implementierung der von BaSys festgelegten Konzepten, ist Eclipse BaSyx, ein von BaSys finanziertes Middleware, das wiederverwendbaren Industrie 4.0 Komponenten und SDK in verschiedenen Programmiersprachen bereitstellt, wobei die schnelle Entwicklung neuer Software-Komponenten ermöglicht wird [4]. Simulink zusammen mit BaSyx könnten eingesetzt werden, um *Hardware-In-The-Loop*, bzw. *Software-In-The-Loop* leichter einzusetzen, wobei BaSyx um die Struktur der digitalen Zwillinge kümmern würde, während Simulink für die Modellierung des Verhaltens einzelner Bauteile oder Regelung gesamter Komponenten angewandt würde.

Nach Prüfung von verschiedenen Alternativen, Simulink und BaSyx zusammen einzusetzen, hat sich herausgestellt, dass eine Kommunikationsschnittstelle zusätzliche Kenntnisse im C, REST und die innere Funktionsweise vom BaSyx voraussetzt, was ein limitierender Faktor für eine breitere Verwendung in der Industrie darstellen könnte. Eine unkomplizierte und erfolgreiche Verbindung der beiden Plattformen würde dem Stand der Technik folgende Vorteile bringen:

- Mehr Anreiz, früher im Projekt nach BaSys strukturierte digitale Zwillinge einzusetzen
- vereinfachte und schnellere Anbindung eines zu testen Hardware/Software an ein MATLAB/Simulink-Modell;
- Reduzieren der Lernkurve, Ingenieure in Simulink mit BaSyx verträglichen Modellen herstellen zu können;
- wirksame Einsetzung realer Daten in MATLAB/Simulink/Modellen.

1.2 Zielstellung

Eine mögliche Lösung für die oben besprochene Problemstellung besteht in einer Simulink-Bibliothek, die in der Lage ist, Verbindungen mit BaSyx-Servern zu realisieren und das Abrufen und Aktualisieren von AAS-Eigenschaften in Echtzeit direkt in dieser Umgebung zu ermöglichen.

Dieses Projekt sieht die Entwicklung der oben beschriebenen Bibliothek und eines Anwendungsbeispiels, das im Wesentlichen aus einem Simulink-Modell besteht, das die entwickelte Bibliothek in Zusammenhang mit einem AAS nutzt. Unten wird der vorgesehene Umfang zusammengefasst:

1. Entwicklung einer Bibliothek für MATLAB/Simulink, um die Verbindung mit BaSyx AAS zu vereinfachen.
2. Einsetzung der Bibliothek in einem Anwendungsbeispiel, das alle Anforderungen aus der Tabelle 1.1 erfüllt.

id	Anforderung
A10	Soll mit einem BaSyx-Server verbinden können

Tabelle 1.1: Anforderungstabelle

Laut der Anforderung A10

1.3 Gliederung der Arbeit

Im Kapitel 2 wird was diskutiert.

Im Kapitel 3 wird was diskutiert.

Im Kapitel 4 wird was diskutiert.

Im Kapitel 5 wird was diskutiert.

Kapitel 2

Theoretische Grundlagen

In diesem Kapitel werden einige Grundlagen erläutert, die für das Verständnis der Arbeit hilfreich sind. Die Arbeit ist eingeteilt in...

2.1 Der Simulationsprozess

Unter Simulation wird die Nachahmung der Funktionsweise eines realen Systems verstanden. Im Kerne der Simulation steht ein Modell, Konstrukt, das die für die Anwendung wichtigsten Merkmale des ausgewählten Systems darstellt.

Sie finden Anwendung in unterschiedliche Bereiche der Technik und nehmen eine wichtige Rolle bei der Lösung dynamischen und komplexen Problemen, die mittels analytischer Vorgehensweise nicht ohne weiteres gelöst werden können. Durch ihren Einsatz gewinnt der Nutzer Klarheit und Verständnis an dem Entwicklungsprozess [5].

In den folgenden Unterkapiteln werden *Hardware-in-the-loop* und *Software-in-the-loop* vorgestellt, zwei Prozessabläufe der Simulationstechnik, die im Produktentstehungsprozess häufig verwendet werden. Der Begriff *x-in-the-loop* deutet darauf hin, dass Abstraktionen eines Systems im Laufe des Entwicklungsprozesses eingesetzt werden, bzw. die Anbindung einer Hardware, sowie eine Software-Darstellung des Systems an die Simulationsumgebung.

2.1.1 Hardware-in-the-Loop (HiL)

Hardware-in-the-Loop (HiL) Ein reales System durch Einsatz von physischen Prototypen ist oftmals wegen den mehreren Konfigurationsmöglichkeiten nicht möglich. Eine Alternative dazu stellt die Anbindung der Hardware an die Simulationsumgebung durch eine elektrische Schnittstelle [5]. Dieser Prozess wird als *Hardware-in-the-Loop* (HiL) genannt. „Hardware-in-the-Loop (HiL) Ansatz bezeichnet oftmals den Einsatz eines realen Steuergerätes oder eines realen Bauteils in einem virtuellen Fahrversuch, ohne dabei ein Risiko für den Fahrer oder Schäden an Prototypen einzugehen.“ ([5])

2.1.2 Software-in-the-Loop

Ein Nachteil des *Hardware-in-the-Loop* wäre der relativ Verzug, bis im Projekt die Hardware verfügbar wird [5]. Eine Alternative für das Testen in früheren Phasen ist die Erstellung einer Software-Darstellung des Systems, die, die für die Phase notwendigen Anforderungen abdeckt. Damit lässt sich eine reine Software-Schnittstelle zwischen den Testkomponenten verwenden, im Gegensatz zu einer aufwendigeren elektrischen Schnittstelle, was für das *Hardware-in-the-Loop* der Fall ist [5].

2.2 MATLAB/Simulink

MATLAB ist ein leistungsfähiges Programm, um numerische Berechnungen in Ingenieurbereich. Diese Software findet Anwendung in der Berechnung, Modellierung und Simulation technischer Systeme und wird sowohl in unterschiedlichen Branchen der Industrie, als auch in der Universität eingesetzt. MATLAB bietet die Grundlage, um mathematische Berechnungen, Datenanalyse, Modellbildung und die Entwicklung von interaktiven Anwendungen zu realisieren [6].

MATLAB setzt sich aus mehreren *toolboxes* zusammen, die jeweils bestimmte Bereiche der Ingenieurwissenschaften abdecken. Eine der bekanntesten *toolboxes* ist Simulink, die eine Blockdiagramm-Umgebung für Multi-Domäne-Simulationen und modellbasierten Entwurf bietet. Diese *toolbox* und der obere Begriff *Model-Based Systems Engineering* werden in dem nächsten Unterkapitel besser erläutert.

2.2.1 Model-Based Systems Engineering (MBSE)

Modellbasiertes Systems Engineering (engl.: *Model-Based Systems Engineering*, MBSE) ist die „formalisierte Anwendung von Modellbildung, um die Aktivitäten der Anforderungserfassung, Entwicklung, Verifikation und Validierung eines Systems, beginnend von der Konzeptphase, über die Entwicklungsphase bis zu späteren Lebenszyklusphasen zu unterstützen“ ([7]).

Modellen dienen zur vereinfachten Darstellung von komplexen Szenarien und können in Lauf des Entwicklungsprozesses eingesetzt werden, wobei oftmals der reine Einsatz von Dokumenten zur Undeutlichkeit und Verkomplizierung eines Systems führen kann. Sie finden häufig Anwendung bei der Anforderungserfassung, Entwicklung, Verifikation und Validierung eines technischen Projektes [7].

2.2.2 Simulink

Simulink ist eine Blockdiagramm-Umgebung für die Auslegung von Systemen mit Mehrdomänenmodellen, Simulationen vor der Umstellung auf Hardware und Bereitstellung des Systems. Mit diesem Werkzeug werden der Entwurf auf Systemebene, die Simulation, die automatische Codegenerierung sowie den kontinuierlichen Test und die Verifikation eingebetteter Systeme unterstützt [2]. Daher lassen sich MBSE und wichtige Aufgaben des Produktentstehungsprozesses wie Architekturwahl, detaillierter Entwurf, die Implementierung und das Testen durch dieses Werkzeuges realisieren [2].

Simulink bietet einen grafischen Editor, anpassbare Blockbibliotheken und Solver für die Modellierung und Simulation dynamischer Systeme. Dieses Werkzeug ist in MATLAB integriert, sodass das Einbinden von MATLAB-Algorithmen in Modelle und das Exportieren von Simulationsergebnissen zur weiteren Analyse nach MATLAB unterstützt werden [2].

Der fundamentale Baustein des Simulink ist der Block (Abb. 2.1), der aus einem Satz von Eingaben, Ausgaben, Zuständen und Parametern besteht.

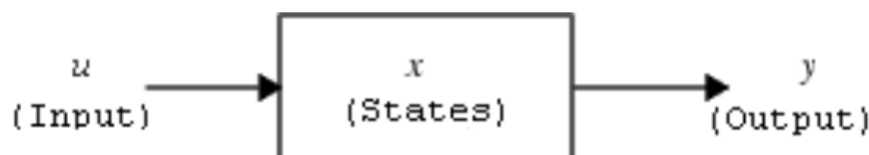


Abbildung 2.1: Struktur eines Simulink-Blockes [8]

Blocks können miteinander verbunden und verschachtelt werden, wobei das Abbild eines Systems sofort sichtbar wird und sein Verhalten sich direkt in der Software in jedem Moment durch Testen validieren lässt.

2.2.3 MATLAB S-Functions

S-Functions (System-Funktionen) sind ein leistungsfähiger Mechanismus zur Erweiterung der Fähigkeiten der Simulink-Umgebung. Eine *S-Function* ist eine in einer der unterstützten Programmiersprachen von MATLAB (MATLAB, C, C++ oder Fortran) geschriebene Beschreibung eines Simulink-Blocks in Computersprache [8].

Sie werden in jedem Simulationsschritt ausgeführt und können in unterschiedlichen Momenten angewandt werden, z.B. an der Initialisierung, Aktualisierung und Terminierung, sowie bei Eingeben- und Ausgeben-Event eines Blockes. Damit lässt sich in Simulink personalisierte Verhalten einbetten, wie z.B. die Verbindung mit anderen Plattformen, wodurch der Austausch vom Parametersatz ermöglicht wird.

2.2.4 Personalisierung der Benutzeroberfläche eines Simulink-Blockes

2.3 Industrie 4.0

Industrie 4.0 (I4.0) ist die bevorstehende vierte große Veränderung in der Fertigung. Sie wurde durch die fortschreitende Digitalisierung ausgelöst und befasst sich mit den Erfordernissen einer wandelbaren Produktion in einer sich immer schneller verändernden Welt [9].

Industrie 4.0 ermöglicht sowohl eine effizientere als auch eine flexiblere Produktion. Die Automatisierung aktueller Produktionssysteme wird häufig mit speicherprogrammierbaren Steuerungen (SPS) realisiert. Diese automatisieren sich wiederholende Aufgaben, die in zyklischen Programmen ausgeführt werden. Sie führen eine Reihe von Fertigungsschritten für jedes Werkstück aus, z. B. eine Abfolge von Fräs- oder Bohrvorgängen. Später kamen weitere Systeme hinzu, z. B. Roboter mit eigenen Betriebssystemen, aber auch ERP-Systeme (*Enterprise Resource Planning*) und MES (*Manufacturing Execution Systems*), um das Ressourcenmanagement zu verbessern und eine flexiblere Produktion von Produktvarianten zu ermöglichen [9].

Die vierte industrielle Revolution ist die durchgängige Digitalisierung der Produktion. Dabei geht es nicht um die Substitution von SPS oder die Einführung neuer Geräte, sondern um neue Systemarchitekturen, die eine Peer-to-Peer-Kommunikation ermöglichen, um beispielsweise eine vorausschauende Wartung, eine KI-basierte Anomalieanalyse in der Produktion und eine effiziente Produktion kleiner Losgrößen zu unterstützen. Bei Industrie 4.0 geht es also nicht um die Integration einer einzigen neuen Technologie. Vielmehr geht es um die Einführung neuer Architekturparadigmen, die eine flexiblere und wandelbarere Produktion ermöglichen, die künftige Wettbewerbsvorteile bringen wird [9].

Der *Industrie 4.0 Maturity Index* (Abb. 2.2) ist ein Werkzeug, um den Entwicklungsgrad eines Unternehmens einzuschätzen. Dieser besteht aus 6 Entwicklungsstufen, wobei jede Stufe auf der vorhergehenden aufbaut und die Fähigkeiten beschreibt, die zur Erreichung dieser Stufe erforderlich sind, sowie die daraus resultierenden Vorteile für das Unternehmen [10].

2.3.1 BaSys

BaSys 4.0 ist ein Plattform-unabhängiges modulares System basiert auf die Struktur der Referenzarchitektur RAMI4.0. Damit wird die Realisierung eines Grundlagersystems für die Produktionsanlagen angestrebt und die folgenden Vorteile abgezielt:

- Bereitstellung und Umsetzung der primären Industrie 4.0-Konzepte;
- einfache dynamische Produktionsänderungen;
- einfache Realisierung digitaler Zwillinge über definierte Schnittstellen;
- vorausschauende Wartung - Online-Zugriff auf die Prozessdaten;
- einfache Integration bestehender und neuer Geräte [11].

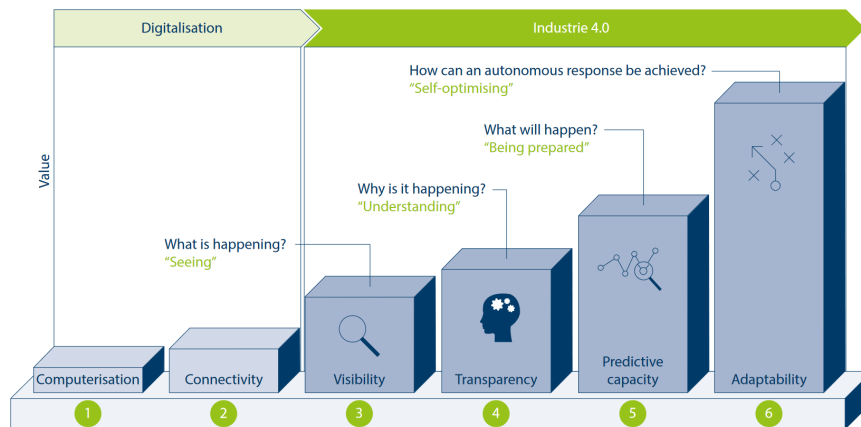


Abbildung 2.2: Stufen des Industrie 4.0-Entwicklungspfades (FIR e. V., RWTH Aachen University [10])

Die Realisierung von BaSys 4.0 basiert auf drei Haupttechnologien, die als zentrale Säulen der Industrie 4.0-konformen Produktionsarchitektur bezeichnet werden: der virtuelle Automatisierungsbuss (VAB), die Verwaltungsschale (AAS), und die Steuerungskomponenten (SPS) [11].

Im Abb. 2.3 werden die Hauptkomponenten eines BaSys4.0-Projektes dargestellt.

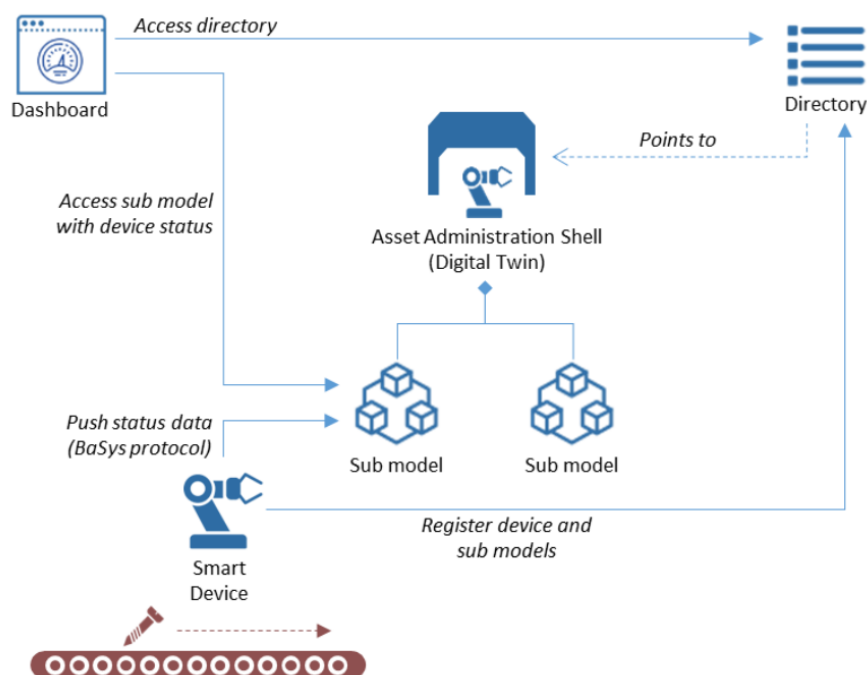


Abbildung 2.3: Beispiel einer einfachen BaSys-Architektur [4]

Verwaltungsschale (AAS) / digitaler Zwilling Die Verwaltungsschale (in Englisch *Asset Administration Shell*, AAS) oder digitaler Zwilling stellt das digitale Abbild eines physischen Gegenstands dar und stellt die Schnittstelle für die Kommunikation in der digitalen Umgebung zur Verfügung [12].

Jedes relevante Gut, z. B. Geräte, Arbeitskräfte, Produkte, aber auch der Herstellungsprozess, enthält genau einen AAS. Das AAS wird durch einen eindeutigen Bezeichner identifiziert, der genau dieses Asset repräsentiert. Aufgrund dieser Vielfalt an repräsentierten Objekten enthält jedes AAS einen unterschiedlichen Satz von *submodels*. *Submodels* sind Datenmodelle, die bestimmte Aspekte einer Anlage beschreiben, z. B. ihr digitales Typenschild, die angebotenen Dienstleistungen oder die Anlagentopologie. Ein AAS und seine *submodels* sind eine einheitliche Darstellung einer realen Anlage [4].

Im Abb. 2.4 werden den Hauptelementen eines AAS dargestellt, bzw. das Asset, die *submodels* und ihre Eigenschaften (*properties*), und das Ganze in derselben einheitlichen Umgebung eines AAS repräsentiert.

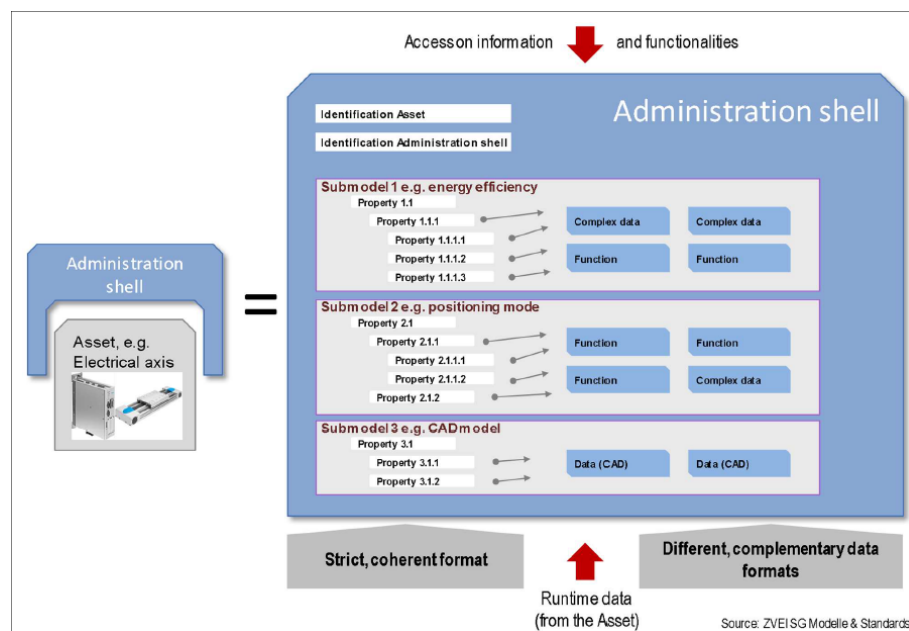


Abbildung 2.4: Abbild eines AAS [4]

Automatisierungsbuss (VAB) Der BaSys 4.0 *virtual automation bus* (VAB) ermöglicht die direkte Kommunikation über die verschiedenen Schichten der Automatisierungsstruktur. Geräte in der Fertigung können mit ERP-Systemen (*enterprise resource planning*) interagieren, selbst wenn die anderen Protokollen anwenden [3].

Der Vorteil dieses Ansatzes ist ein reduzierter Aufwand bei der Realisierung unterschiedlicher Kommunikationstechnologien. Anstatt von einer Übersetzung jedes einzelnen Protokolls wird es dadurch möglich die Verwendung von Primitiven, um diesen Prozess zu schichten und die Komplexität zu reduzieren [4].

Im Abb. 2.5 wird es ein Beispiel-Einsatz von einem VAB dargestellt.

2.3.2 Eclipse BaSyx

Eclipse BaSyx ist eine Open-Source-Referenzimplementierung von BaSys 4.0-Konzepten, die gängige Industrie 4.0-Komponenten und ein Software Development Kit (SDK) enthält, das die Entwicklung neuer Komponenten unterstützt. Eclipse BaSyx ist aus dem BaSys 4.0 Projekt hervorgegangen, das vom Bundesministerium für Forschung und Bildung gefördert wird. Mit seiner Anwendung lassen sich folgende Punkte realisieren:

- End-to-End-Digitalisierung der Produktion;

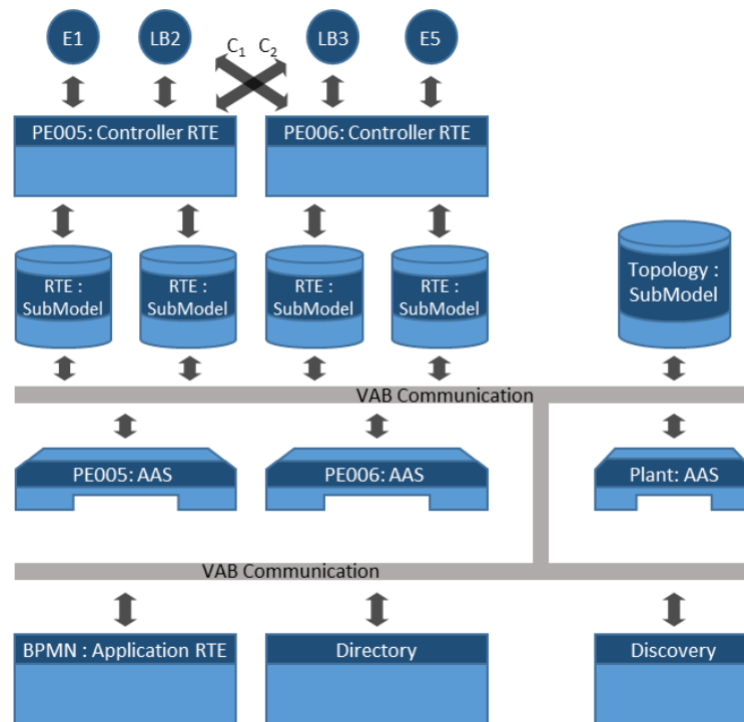


Abbildung 2.5: VAB-Beispiel mit verbundenen Entitäten [4]

- End-to-End-Konnektivität zwischen Werkstatt und IT;
- Peer-to-Peer-Kommunikation zwischen Geräten und der IT;
- digitale Prozessmodelle, Wertströme und Produktverfolgung;
- veränderbare Produktionsprozesse bis hin zur Losgröße-Eins-Produktion;
- Zwillinge für Prozesse, Produkte, Geräte und mehr;
- Big-Data-Analyse von Produktionsprozessen;
- vorkonfigurierte Container für Industrie 4.0-Anwendungen, z.B. Predictive Maintenance, Analysen und Dashboards;
- automatisierte Rückverfolgung und Dokumentation von Produktionsprozessen [4].

Diese Implementierung wird in 4 Ebenen umgesetzt:

1. Feldebene: Die Feldebene besteht aus Automatisierungsgeräten, Sensoren und Aktoren ohne eine spezifische BaSys-konforme Schnittstelle;
2. Geräteebene: Die Geräteebene besteht aus Automatisierungsgeräten, die eine BaSys 4.0-konforme Schnittstelle anbieten;
3. Middleware-Ebene: Die Middleware-Ebene besteht aus wiederverwendbaren Industrie 4.0-Komponenten, die notwendige generische und anlagen-unabhängige Fähigkeiten für Industrie 4.0-Produktionslinien implementieren;
4. Anlagenebene: Die Anlagenebene besteht aus High-Level-Anlagenkomponenten, die die Produktion verwalten, optimieren und überwachen [4].

Im Abb. 2.6 wird eine vereinfachte Struktur, um der Einsatz von den durch BaSyx implementierten Komponenten, e.g. AAS, Registry-Komponente und *submodels*, zu veranschaulichen [4].

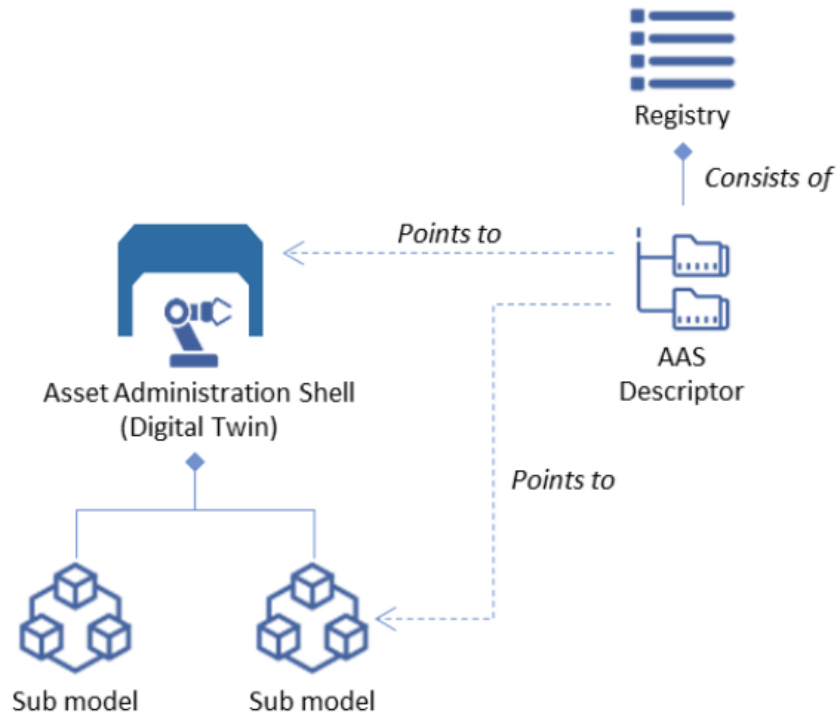


Abbildung 2.6: Diagramm einer Eclipse BaSyx-Systemstruktur [4]

Registry-Komponente Die BaSys 4.0 *registry*-Komponente ermöglicht die Registrierung und Suche von AAS innerhalb definierter Systemgrenzen. Entitäten, die AAS einschließlich ihrer Submodelle anbieten, können sich registrieren und damit anderen Teilnehmern ermöglichen, sie zu finden. Die für die Registrierung erforderlichen Informationen sind u.a. eindeutige Identifikatoren für AAS und ihr jeweiliges Asset, Endpunktinformationen zur API der AAS und deren *submodel*-Dienstleister [4].

AAS-Server-Komponente Die AAS-Server-Komponente stellt einen leeren AAS-Server zur Verfügung, auf dem mehrere AAS und *submodels* gehostet werden können [4].

2.4 REST-Schnittstellen

REST ist der am meisten verbreitete Kommunikationsarchitektur zur Vernetzung von Maschinen und wird bei mehreren Anwendungsfällen angewandt, wie z.B. IoT-Technologien [13]. In diesem Zusammenhang werden zwischen zwei Entitäten unterscheiden, nämlich der Server und der Client. Dieser initialisiert die Anfrage an den ersteren, der anschließend diese bearbeitet und dem Client zurückgibt [14].

In REST werden Daten über HTTP übertragen und wird es dadurch die Realisierung von Webservices ermöglicht. Ressourcen (jedes Objekt einer Anwendung) repräsentiert eine Ressource, die über eine Adresse (URI, eng.: *Uniform Resource Identifier*) zugreifbar wird [14]. Auf diese Ressource kann eine Anwendung zugreifen, wenn ihr den Identifikator, die zu auszuführende Aktion und das Format der erwartenden Antwort des Servers bekannt sind [14]. Allgemein gilt, dass jede über HTTP übertragene Botschaft alle Informationen enthalten darf, die für ihr volle Assimilierung in der

Serverseite gebraucht werden, und daher wird die gleiche Anforderung von der REST-Schnittstelle geerbt.

In dieser Architektur werden vier generische HTTP-Methoden zur Verfügung gestellt: GET, PUT, POST und DELETE. Diese werden jeweils verwendet, um das Aufrufen, das Hinzufügen, Verarbeiten und das Löschen von Daten des Servers auszuführen [14].

Kapitel 3

Plannung und Durchführung

3.1 Entwicklungsprozess

Dieser Kapitel erläutert in mehr Detail, wie das System ausgegliedert ist und wie das Projekt entworfen wurde. Im Kapitel 3.1.2 wird das Konzept erläutert, und die für dessen Realisierung notwendigen Anforderungen aufgeführt. Den funktionalen Aspekten des Projektes finden sich im Kapitel 3.1.3 veranschaulicht.

3.1.1 Anforderungen

Die Entwicklung der Bibliothek hat vorwiegend zwei Anwendungsfälle berücksichtigt:

1. AF1: als Schnittstelle zum Testen von einer Hardware/Software über Simulink im Abbild-Modus, wobei die Simulation mit einem am Anfang aufgenommenen Abbild der Parameterwerte arbeitet, indem das Lesen von Parametern einmal am Anfang der Simulation und Setzen einmal am Ende dieser stattfindet;
2. AF2: als Schnittstelle zum Testen von einer Hardware/Software über Simulink mit verlangsamter Simulationstaktung. Dies ermöglicht das in Echtzeit Lesen und Setzen von Parametern des digitalen Zwillings im Laufe der Simulation. Die Abtast-, bzw. Aktualisierungsrate können im Block eingestellt werden.

Aus den Anwendungsfällen lässt es sich zwei Betriebsmodi entnehmen: Abbild-Modus für den ersten Anwendungsfall und Betriebsmodus 2: Echtzeitmodus für den zweiten.

Für die ordentliche Realisierung der obenerwähnten Anwendungsfälle setzt sich voraus, dass das System in der Lage sein soll:

1. A1: den Wert aus einem SubmodelElement abzulesen
2. A2: den Wert aus einem SubmodelElement zu aktualisieren
3. A3: aus einem Registry Server Daten vom einen AAS abzurufen
4. A4: Fehler in Simulink-konformen Fehlermeldungen anzuzeigen
5. A5: Parallelität angemessen zu implementieren, indem die Verbindung mit mehreren *SubmodelElements* in einem gleichen Modell möglich ist.
6. A6: beide Betriebsmodi (Abbild- und Echtzeitmodus) zu realisieren.
7. A7: eine Bedieneroberfläche im Simulink zu haben, mit der sich Parameter wie Betriebsmodus, ID von *AAS*, *Submodel*, und *SubmodelElement* einstellen lassen.

In diesem Entwicklungsschritt wurden keine Anforderungen bezüglich Grenzen für die Abstrakte von beiden Lese- und Schreibverfahren und Sicherheit der Datenübertragung festgelegt, stattdessen liegt der Schwerpunkt der Arbeit an der Entwicklung einer ausbaubaren Grundlage für die Bibliothek, die eventuell durch feinere Anforderungen permissiv erweitert werden könnte.

3.1.2 Konzept und Anforderungsdefinition

Die Lösung besteht in einer Bibliothek für Simulink, die mit einem vorhandenen BaSyx-AAS-Server verbindet und das Ablesen und Aktualisieren von dessen Elementen ermöglicht. Dafür wurde auf die verfügbare REST-Schnittstelle vom BaSyx und Matlab S-Functions zugegriffen. S-Functions ergänzen die d Simulink-Umgebung, indem personalisierten Blöcke in einer der unterstützenden Programmiersprachen (z.B. C, C++, Fortran und Matlab) entworfen werden können.

3.1.3 Funktionale Spezifikation

Für die Modellierung des funktionalen Anteils des Systems wurde die Arcadia-Methodologie mit der Capella-Modellierungsumgebung verwendet. Das System lässt sich in 3 Teilsystemen unterteilen: der (1) *BasyxConnector*, der (2) *HttpClient* und der Blöcke (3) *PropertySAConnectorBlock*.

Der *BasyxConnector* dient als Kommunikationsschnittstellen zwischen einem Block und einem Client. Genauer gesagt werden durch den die sämtlichen für eine Client-Anfrage notwendigen Informationen generiert und diese an den intern-gespeicherten Client weitergeleitet. Darüber hinaus wird die Logik für die Kommunikation mit einem BaSyx-AAS-Server, bzw. Basyx-Registry-Server, in dieser Komponente integriert. Für den Umfang dieses Projekt wurde nur eine Art von *Client* entworfen, bzw. die *HttpClient*, dessen Aufgabe aus sowohl der Abgabe von Http-Anfrage wie *GET*, *PUT*, *POST* und *DELETE*, als auch dem Empfangen der Antwort seitens des Servers besteht.

Außerdem wurden nur Open-Source Bibliotheken angewendet, beziehungsweise die *libcurl* und *jsoncpp*. Das Erstere wird effektiv als Implementierungsgrundlage für den *HttpClient* angewendet, während das Letztere zur Dekodierung von Http-Antworten dient.

Der Block nimmt die Funktionalitäten eines *CommonBlock* im Anspruch, wodurch die Ein- und Ausgabe von Werten an *Simulink* sich implementieren lassen. Partikulär vom *PropertySAConnectorBlock* ist ein Funktionsbaustein, um Leseanfrage und Aktualisierungsanfrage an einen eingegebenen Submodel Element mittels den *BasyxConnector* zu tätigen.

Darüber hinaus wurde eine zusätzliche Komponente zur Bearbeitung von URLs entwickelt. Diese Komponente wird durchaus im Projekt verwendet und verfügt über mehrere Methoden, URL automatisch in seinen Bauteilen zu zerlegen und dessen API-Pfad zu bearbeiten.

Ein normaler Kommunikationsfluss besteht darin, dass der Block *PropertySAConnectorBlock* (3) in Simulink ausgeführt wird, wodurch nun Methoden vom *BasyxConnector* aufgerufen werden. Dieser setzt im Anschluss den *HttpClient* ein, um effektiv die Http-Anfrage auszusenden. Die Antwort des Servers wird im Anschluss vom *BasyxConnector* abgewickelt und entweder den abgerufenen Wert oder eine Fehlermeldung dem ursprünglichen Block weitergegeben.

Mögliche Events sind das Start-, Takt- und Schluss-Event. Für das Start-Event werden beispielsweise alle durch die Simulink-Oberfläche eingegebenen Parameter im Block internen Speicher abgespeichert. Das Takt-Event wird zu jedem Simulink-Zeitschritt aufgerufen, wobei die aktualisierten Eingabewerte an dem Block geschickt werden und wiederum eine aktualisierte Ausgabe vom Simulink erwartet wird. Im Schluss-Event sollen grundsätzlich für diesen Anwendungsfall alle initialisierten Objekten vom Arbeitsspeicher bereinigt werden.

In der Abb. 3.1 lassen die Funktionsbausteine der oben erwähnten Komponenten sich veranschaulichen.

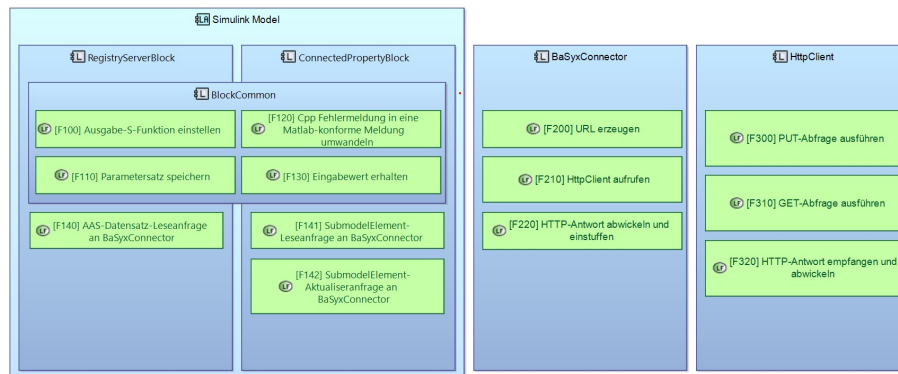


Abbildung 3.1: Funktionsbausteine jeder Teilkomponente

3.1.4 Systemarchitektur

Logische Architektur für das Erfüllen vom A1, A2 und A4

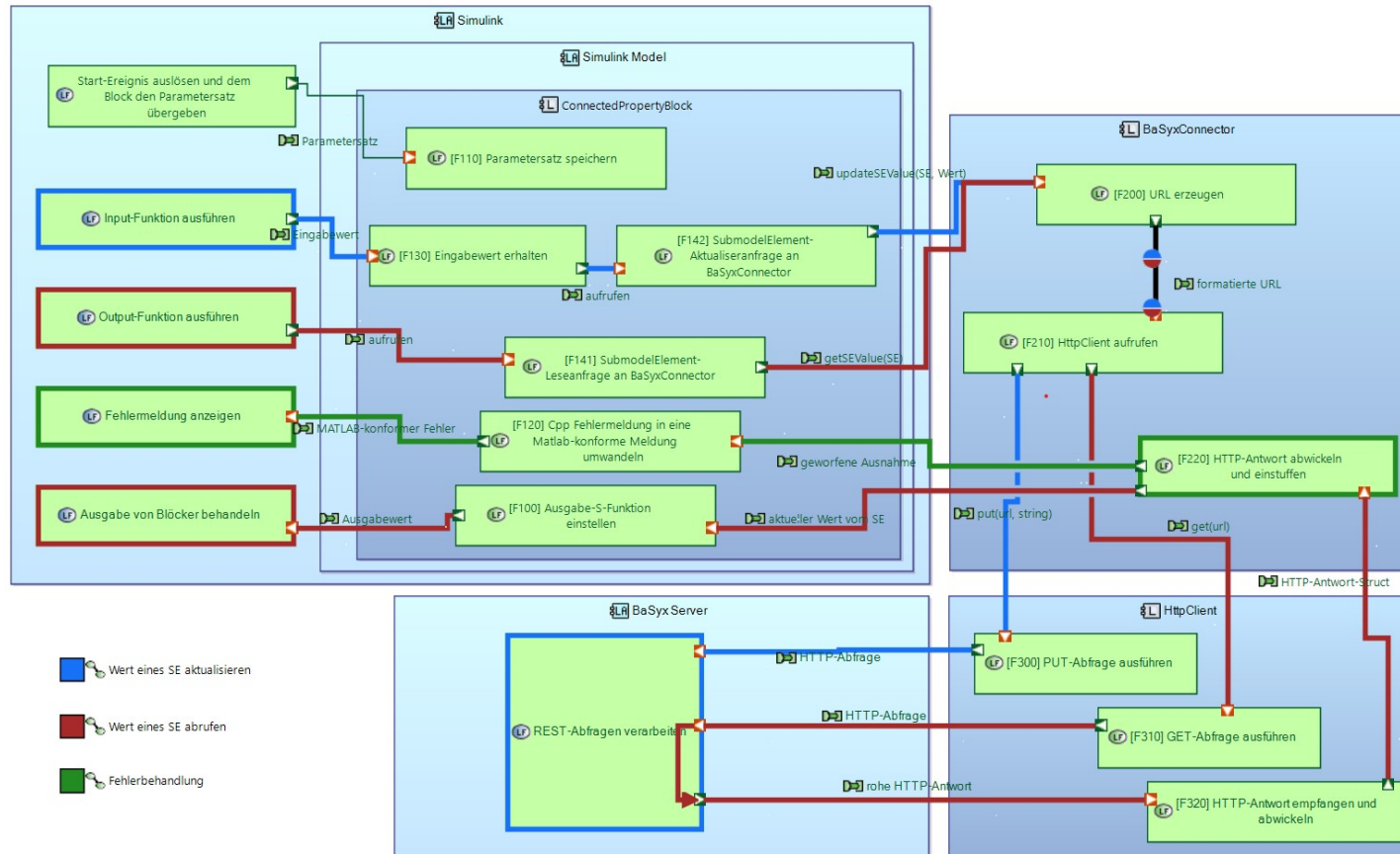


Abbildung 3.2: Darstellung des Kommunikationsflusses vom A1, A2 und A4

Klassendiagramm

Im Diagramm 3.3 lässt sich ein Überblick der Architektur der Bibliothek erhalten. Zusätzlich zu den internen Komponenten (`BasyxConnector`, `httpClient`) und den externen Bibliotheken (`jsoncpp`, `curlcpp`, und `simstruc`) wird die Komponente `UrlHandler` im Anspruch genommen, wobei die für die Bearbeitung von URL und Zergliedern von `strings` verantwortlich ist.

Wie in den vorherigen Kapiteln erklärt wurde, wird *curlcpp* für das Tätigen HTTP-Anfragen und *jsoncpp* für die Dekodierung dieser verwendet. Außerdem legt *simstruc* die Datenstruktur für Simulink fest und kapselt die auf das Modell, bzw. *S-Function* bezogenen Parameter und Ausgaben ein.

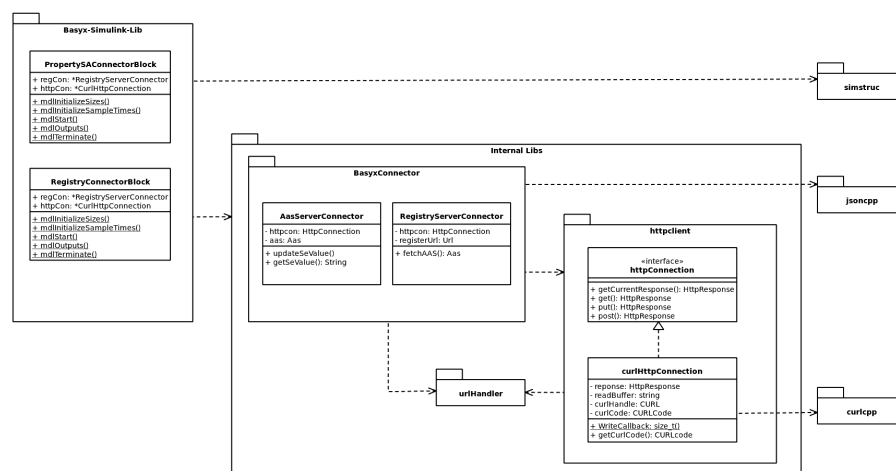


Abbildung 3.3: Klassendiagramm

3.2 Entwurf

Dieser Kapitel fängt mit einer Erläuterung davon an, was für Werkzeuge für die Entwicklung des Systems angewendet worden sind. Anschließend wird es auf die Einkapselung von BaSyx in einer S-Funktion und jede im System implementierte Fähigkeit eingegangen, wobei jeweils das Konzept und Lösungsansatz kurz erklärt wird. Zum letzten Thema ist einen Überblick über den Entwurf direkt in der Simulationsumgebung vom Simulink gegeben, mit dem Schwerpunkt auf die Personalisierung der Benutzerschnittstelle eines mit S-Funktion erstellten Blockes.

3.2.1 Entwicklungsumgebung

Die Bibliothek wurde in C++ mit Visual Studio Code als integrierte Entwicklungsumgebung entwickelt. Zusätzlich war CMake durchaus eingesetzt, um das *Packaging* der sämtlichen Modulen durchzuführen. Für alle Modulen wurde testgetriebene Entwicklung eingesetzt, um sicherzustellen, dass alle funktionelle und nicht-funktionelle Anforderungen abgedeckt werden. Darüber hinaus wurden beide *google-test* und *google-mock* für die Realisierung von Testfällen angewendet.

Für die Ausführung der Testszenerien sind eine Kombination aus Docker und Simulink verwendet worden, wobei die von BaSyx veröffentlichten Docker-Containers für den Registry- und AAS-Server für die Bereitstellung der BaSyx-Architektur eingesetzt wurden. Außerdem wurden die Verwaltungsschale mittels *AASX Package Explorer* erstellt, eine Open-Source Software zur grafischen Visualisierung und Bearbeitung von *AAS/Submodel/SubmodelElements*.

3.2.2 Wrappen des BaSyx als S-Funktion

Der *BasyxConnector* wird innerhalb eines Blockes direkt aufgerufen, um Operationen wie einen AAS aus dem Registry-Server abzurufen und den Wert eines *SubmodelElements* anzufragen, bzw. aktualisieren.

Ein Öffner der Klasse *aasServerConnector* zeigt auf, dass diese im Konstruktor beide ein Objekt des Typs *AAS* und *HttpConnection* erfordert. Zum Aufbau eines AAS-Objekts wird es auf die Datei *components.hpp* verwiesen, wobei eine neue Instanz entweder aus der Klasse *registryServerConnector* über die Methode *fetchAAS(...)* ausgegeben werden kann, oder anhand der ID und URL-Pfades erstellt werden kann.

Ein *HttpConnection*-Objekt kann über eine der Konstruktion einer der Implementierungen der Schnittstelle *httpConnection.hpp* angelegt werden. Diese dient zu dem Tätigen von HTTP-Anfragen, wodurch eine effektive Entkopplung von deren Ausführung und dem *BasyxConnector* erreicht wird. Momentan wurde nur eine Implementierung (*curlHttpConnection*) über die externe Bibliothek *libcurl* realisiert.

Um eine Änderung der fest codierten API-Pfaden leichter vorzunehmen, wurden alle API-Pfaden in einer einzigen Datei (*basyxConfig.hpp*) eingefügt, auf die die sämtlichen Methoden der Bibliothek zugreifen können.

```
// BasyxConfig.hpp
...
namespace BasyxConfig
{
    using std::string;
    // Registry Server
    static const string REG_API_PATH = "api/v1/registry";

    // AAS Server

    static const string SM_EPP_ = "submodels/"; //+smIdShort
    static const string SE_EPP_ = "/submodel/submodelElements/"; //+seIdShort
    static const string VALUE_EPP_ = "/value";

    static string buildSeEpPath(string smIdShort, string seIdShort)
    {
        string e_smIdShort = escapeStr(smIdShort);
        string e_seIdShort = escapeStr(seIdShort);

        return SM_EPP_+e_smIdShort+SE_EPP_+e_seIdShort+VALUE_EPP_;
    }
};
```

Listing 1: Auszug der API-Definitionen in BasyxConfig

Innerhalb der Datei *PropertySACConnectorBlock.cpp* findet sich die Logik, um eine Verbindung mit beiden *BaSyx* und *Simulink* herzustellen. Die Klasse *CurlHttpConnection* wird als Zeiger instanziiert, und als Argument zur Erstellung von Objekten des Paketes *BasyxConnector* über die Methoden *mdlStart* und *mdlOutputs* verwendet.

Um globale Variablen innerhalb der *Simstruc*-Umgebung zu realisieren, werden *Work* Vektoren benötigt. Mit diesen lässt sich Zustand innerhalb einer Block-Instanz speichern und dadurch Initialisierungsparameter wie *aasId*, *registryUrl*, *submodelElementId* unter anderen beibehalten werden. Dafür werden hauptsächlich zwei *simstruct*-Methoden eingesetzt, nämlich *ssGetPWork*,

ssGetPWorkValue. Die erstere dient zur Bearbeitung des *Work* Vektors, während die letztere zum Ablesen eines in diesem gespeicherten Wert.

Im Auszug 2 lässt sich Beispiele für die Anwendung von *Work* Vektors entnehmen, in dem diese erst über die Methode *initializePVectors* initialisiert werden, und später bei jedem Lesevorgang verwendet, um die *AAS*, *Submodel* und *SubmodelElement* Objekten abzurufen.

```
// PropertySACConnectorBlock.cpp
...
std::unique_ptr<CurlHttpConnection> httpCon;
...
static void initializePVectors(SimStruct *S)
{
    ...
    Aas* aas;
    if(aasUrlStr == ""){
        Url urlStr = Url(regUrlStr);
        RegistryServerConnector regCon = RegistryServerConnector(std::move(urlStr),
        ↪ *httpCon);
        aas = new Aas(regCon.fetchAAS(aasId));
    }else{
        Url urlStr= Url(aasUrlStr);
        aas = new Aas(urlStr, aasId);
    }

    Submodel* sm = new Submodel(*aas,smIdShort);
    SubmodelElement* se = new SubmodelElement(*sm,seIdShort);

    void **PWork = ssGetPWork(S);
    PWork[P_AAS] = aas;
    PWork[P_SM] = sm;
    PWork[P_SE] = se;
}
...
static double readSE(SimStruct *S){
    Aas* aas = (Aas*) ssGetPWorkValue(S,P_AAS);
    SubmodelElement* se =(SubmodelElement*) ssGetPWorkValue(S,P_SE);
    AasServerConnector aasCon = AasServerConnector(*aas, *httpCon);
    return std::stod(aasCon.getSeValue(*se));
}
...
```

Listing 2: Auszug der PropertySACConnectorBlock.cpp Datei

Die Methode *mdlStart* wird am Moment der Instantiierung des Blockes aufgerufen und Objekten wie *AAS*, *Submodel* und *SubmodelElement* werden angelegt und im internen Zustand über einen *work* Vektor gespeichert. für einzelnen Block individueller Zustand Bei jedem Simulationsschritt, wird die Ausgabe über *mdlOutputs* berechnet. An dem Moment unterscheid *PropertySACConnectorBlock*, ob eine Verbindung am Eingang, am Ausgang oder an beiden Ports liegt. Je nach Fall wird an den *BasyxConnector* eine Aktualisierungs-, bzw. Leseanfrage übersendet (s. 3).

```

// PropertySAConnectorBlock.cpp
...
static void mdlOutputs(SimStruct *S, int_T tid)
{
    const double* u = (const double*)ssGetInputPortRealSignal(S, 0);
    double        *y = ssGetOutputPortRealSignal(S,0);
    try{
        // Case snapshot mode
        if(getNumPar(S,PAR_BM)==0){
            y[0] = getLocalSeValue(S);
        }// case real-time mode
        else{
            if (ssGetInputPortConnected(S,0)) updateSE(S,u[0]);
            if (ssGetOutputPortConnected(S,0)) y[0] = readSE(S);
        }
    }catch (const std::exception& e){
        ssSetErrorStatus(S,e.what());
        return;
    }
}
...

```

Listing 3: Auszug der PropertySAConnectorBlock.cpp Datei

3.2.3 A1: Ablesen des Wertes aus einem SubmodelElement

Für das Ableseverfahren wird es vorausgesetzt, dass eine Verbindung an dem Blockausgang vorliegt. Die Methode *mdlOutputs* wird ausgeführt, wohingegen die Methode *getSeValue* des *BasyxConnector* aufgerufen wird. Dieser erzeugt wiederum eine formatierte URL für den *HttpClient*. Anschließend stellt der *HttpClient* eine GET-Anfrage an dem *BaSyx-Server*, wobei dessen Antwort vom *HttpClient* in einem *Struct* mit expliziten Parametern für die rohe Antwort und den *Status-Code* umgewandelt wird.

Dieses *Struct* wird im *HttpConnection* verarbeitet und im Fall eines *Status-Code* gleich 0 ist der Wert des *SubmodelElement* zurückgegeben. Letztendlich setzt der Block den Wert seines Ausgangs zu dem erhaltenen Wert. Das Sequenzdiagramm für den erfolgreichen ausgeführten Prozess lässt sich in der Abb. 3.4 finden.

3.2.4 A2: Aktualisierung des Wertes aus einem SubmodelElement

Für das Aktualisierungsverfahren wird vorausgesetzt, dass eine Verbindung an dem Blockeingang vorliegt. Wie bei dem davor beschriebenen Prozess wird die Methode *mdlOutputs* ausgeführt, wobei die Methode *updateSeValue* des *BasyxConnector* aufgerufen wird. Dieser erzeugt wiederum eine formatierte URL für den *HttpClient*. Anschließend stellt der *HttpClient* eine PUT-Anfrage an dem *BaSyx-Server*. Die Antwort wird letztendlich vom *BasyxConnector* und falls einen Fehler aufgetreten ist, wird eine Ausnahme ausgelöst und der Prozess im 3.2.6 wird gestartet.

In der Abb. 3.5 findet sich das Sequenzdiagramm von dem oben detaillierten Prozess.

3.2.5 A3: Abrufen von AAS-Daten aus einem Registry Server

Im Gegensatz zu dem Ableseverfahren des Wertes eines *SubmodelElement* wird die Logik für das Abrufen eines AAS-Datensatzes in der Methode *mdlStart* (über die Methode *initializePVectors*)

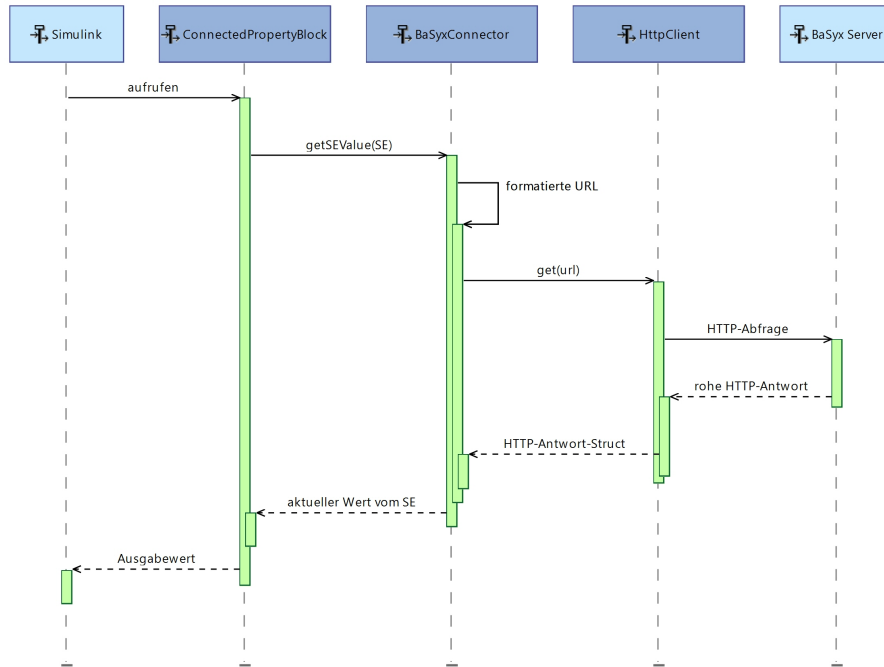


Abbildung 3.4: Sequenzdiagramm für A1

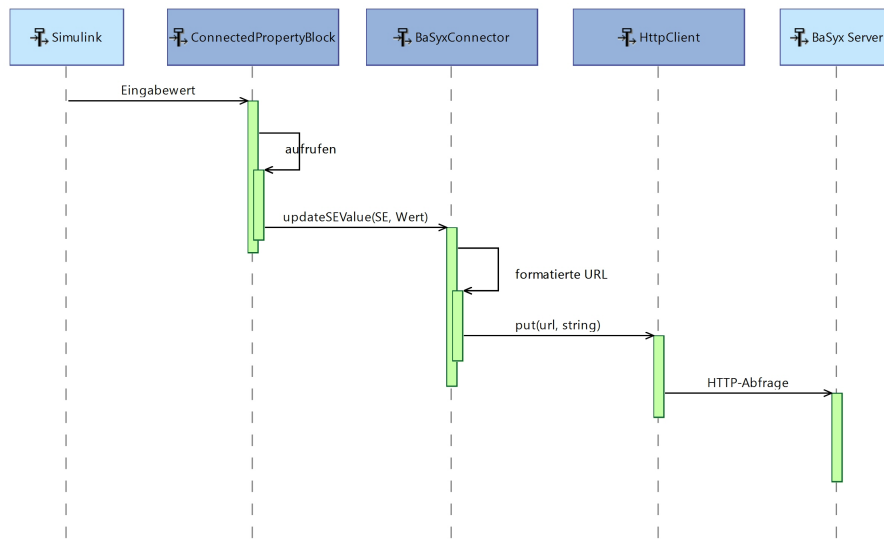


Abbildung 3.5: Sequenzdiagramm für A2

```

// PropertySACConnectorBlock.cpp
...
static double readSE(SimStruct *S){
{
    Aas* aas = (Aas*) ssGetPWorkValue(S,P_AAS);
    SubmodelElement* se =(SubmodelElement*) ssGetPWorkValue(S,P_SE);
    AasServerConnector aasCon = AasServerConnector(*aas, *httpCon);
    return std::stod(aasCon.getSeValue(*se));
}
...

// aasServerConnector.cpp
...
string AasServerConnector::getSeValue(SubmodelElement &se)
{
    ...
    HttpResponse resp = httpCon_.get(aas_.aasUrl);

    if(resp.httpCode != 200 )
        throw std::runtime_error("Unable to get SubmodelElement value at
↪  '"+aas_.aasUrl.getStr()+"'");

    se.updateValue(resp.raw);
    ...
}
...

// httpConnection.hpp
...
struct HttpResponse{
    HttpResponse() : raw(""), httpCode(0) {}
    HttpResponse(string raw, long code) : raw(raw), httpCode(code) {};

    string raw = "";
    long httpCode = 0;
};
...
}

```

Listing 4: Auszug der PropertySACConnectorBlock.cpp Datei

ausgeführt und anschließend wird die formatierte URL über die Methode *fetchAAS* vom *BasyxConnector* erzeugt. Der URL wird am *HttpClient* überreicht, wobei aus der Komponente eine GET-Anfrage an dem textitBaSyx-Server gestellt wird. Die davon erhaltene Antwort ist am Ende vom *BasyxConnector* abgewickelt, und ein Objekt des Typs *AAS* wird am Block zurückgegeben. Der Prozess ist in der Abb. 3.6 veranschaulicht.

3.2.6 A4: Anzeigen von Simulink-konformen Fehlermeldungen

Wäre ein Status-Code nicht gleich 200 erhalten, löst der *BasyxConnector* eine Ausnahme mit Details des empfangenen Paketes aus, die weiterhin an dem entsprechenden Block zugeleitet wird. Für das Wrappen der Ausnahme wird eine Simulink-konforme Methode angewandt, durch die Fehlermeldungen direkt im Simulink-Konsole angezeigt werden können. Das Abbild 3.7 stellt ein

```

// PropertySAConnectorBlock.cpp
...
static void updateSE(SimStruct *S, double inputValue){
    Aas* aas = (Aas*) ssGetPWorkValue(S,P_AAS);
    SubmodelElement* se =(SubmodelElement*) ssGetPWorkValue(S,P_SE);
    AasServerConnector aasCon = AasServerConnector(*aas, *httpCon);
    aasCon.updateSeValue(*se,std::to_string(inputValue));
}
...

// aasServerConnector.cpp
...
void AasServerConnector::updateSeValue(SubmodelElement &se, const string
→ &value)
{
    ...
    if(httpCon_.put(aas_.aasUrl,value).httpCode == 200 )
        se.updateValue(value);
    else
        throw std::runtime_error("Unable to update SubmodelElement value at
→ '"+aas_.aasUrl.getStr()+"'");
}
...
}

```

Listing 5: Auszug der PropertySAConnectorBlock.cpp Datei

```

// PropertySAConnectorBlock.cpp
...
static Aas createAas(string aasUrlStr, string regUrlStr, string aasId){
    // Create AAS based on whether user has provided an AasUrlStr or not
    if(aasUrlStr == ""){
        Url urlStr = Url(regUrlStr);
        RegistryServerConnector regCon = RegistryServerConnector(std::move(urlStr),
→ *httpCon);
        return Aas(regCon.fetchAAS(aasId));
    }else{
        Url urlStr= Url(aasUrlStr);
        return Aas(urlStr, aasId);
    }
}
...

```

Listing 6: Auszug der PropertySAConnectorBlock.cpp Datei

Beispiel einer Fehlermeldung, bei der der Server nicht in der Lage war, mit dem Registry-Server eine Verbindung herzustellen.

Diese ist ein wesentlicher Aspekt der Anwendung, was unnötige Abstürze im Programm vermeidet und den Nutzern ermöglicht, Tippfehler schnell zu erkennen und die Verbindung mit dem BaSyx-Server zu überprüfen.

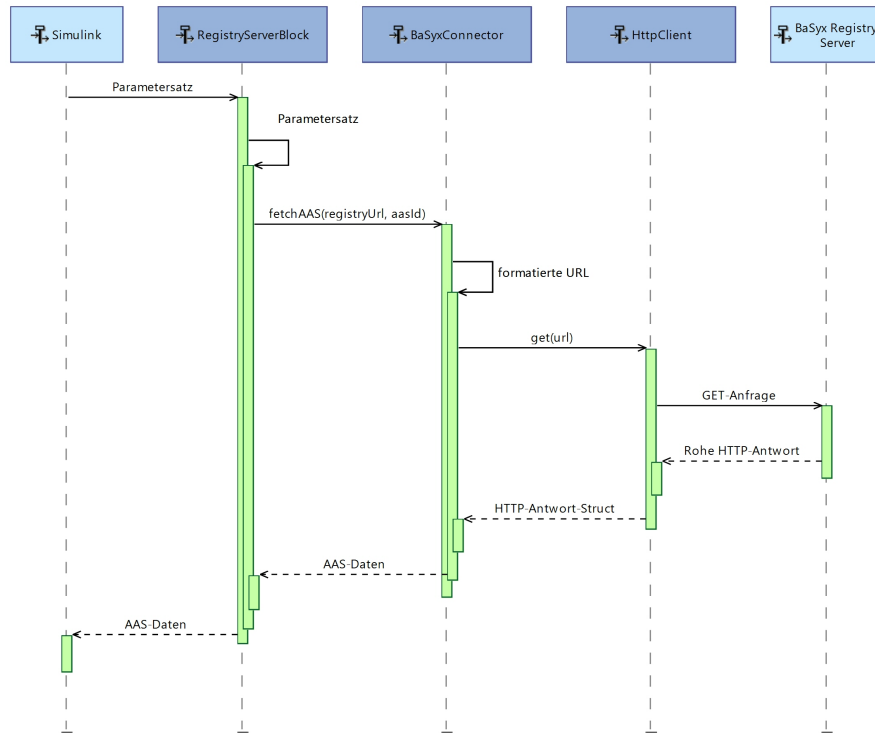


Abbildung 3.6: Sequenzdiagramm für A3

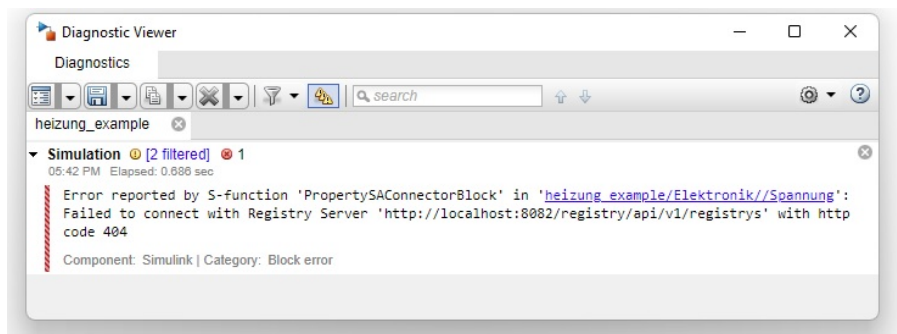


Abbildung 3.7: Beispiel einer Fehlermeldung

3.2.7 A5: Anwendung von mehreren Blöcken in demselben Simulink-Modell

Aus Sicht Matlab/Simulink ist ein mit S-Funktion entwickelter Block eine geteilte Bibliothek. Eine globale Variable ist deshalb mit allen anderen Blöcken gleiches Typs geteilt, was ein Problem für die Beibehaltung der Daten unter Simulationsschritten darstellt. Wie bereits erwähnt, können zur Umgehung dieses Problems und um den Blöcken einen internen Zustand zu verleihen, *work*-Vektoren verwendet werden.

Das Abbild 3.8 zeigt ein Modell mit mehrfach eingefügten Instanzen des Blockes *PropertySABlockConnectorBlock*. Jede Instanz funktioniert unabhängig voneinander, trotz unterschiedlichen Abtastrate und auf *SubmodelElement* bezogenen Parametern.

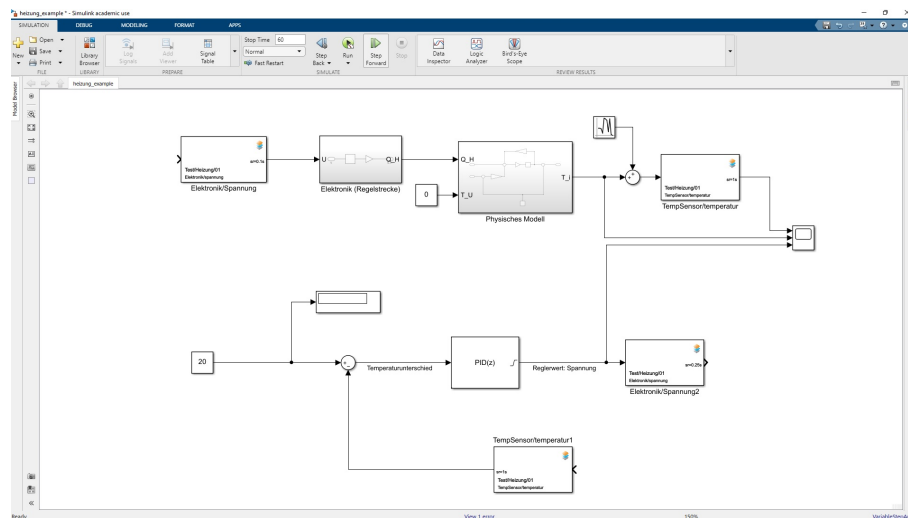


Abbildung 3.8: Simulink-Modell mit mehreren Blöcken

3.2.8 A6: Betriebsmodus wechseln

Der Betriebsmodus ist als Parameter im Simulink eingegeben, und anschließend in den *mdlStart*, *mdlOutputs* und *mdlTerminate* Methoden in "*PropertySABlockConnector.cpp*" geprüft und implementiert. Beim Einschalten vom Abbild-Modus wird die Daten des *SubmodelElement* nur beim Aufruf vom *mdlStart* und *mdlTerminate* abgerufen, während dies im Echtzeitmodus bei jedem Aufruf von *mdlOutput* geschieht.

3.2.9 A7: Personalisierung der Blockbenutzeroberfläche

Wie im Kapitel 2.2.4 kurz erläutert, bietet Simulink mehrere Möglichkeiten der Benutzeroberfläche eines Blockes zu personalisieren.

Bei beiden Blöcken war es eine kritische Anforderung, dass alle einzustellenden Parameter gut erkennbar waren, um schnell von mehrfachen eingefügten Blöcken zu unterscheiden.

Das Block-Symbol wurde personalisiert, um schnell einen Überblick der eingestellten Parameter zu bekommen. Dazu sind Informationen wie die Abtastrate und das verbundene *SubmodelElement* und dessen *Submodel*, bzw. *AAS* immer sichtbar, wie im Bild 3.9 dargestellt ist. Das ermöglicht die schnelle Erkennung und Unterscheidung von mehrfach eingefügten Blöcken in einem Modell.

Nach Tippen auf einen Block wird dem Benutzer ein Dialogfenster mit Textfelder für alle einzustellende Parameter angezeigt. Das wurde erst durch die Erstellung einer Maske für die S-Funktion, eine im Simulink eingebettete Funktionalität. Das Abbild 3.10 zeigt ein Beispiel einer Instanz des Blockes *PropertySABlockConnectorBlock*.

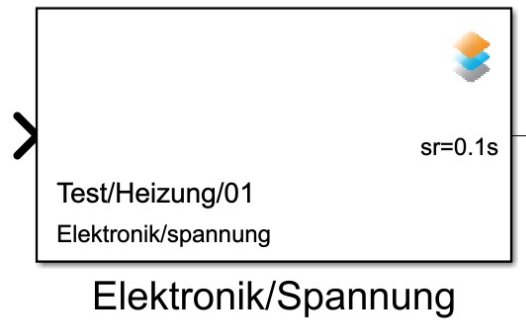


Abbildung 3.9: Instanz eines PropertySACconnector-Block

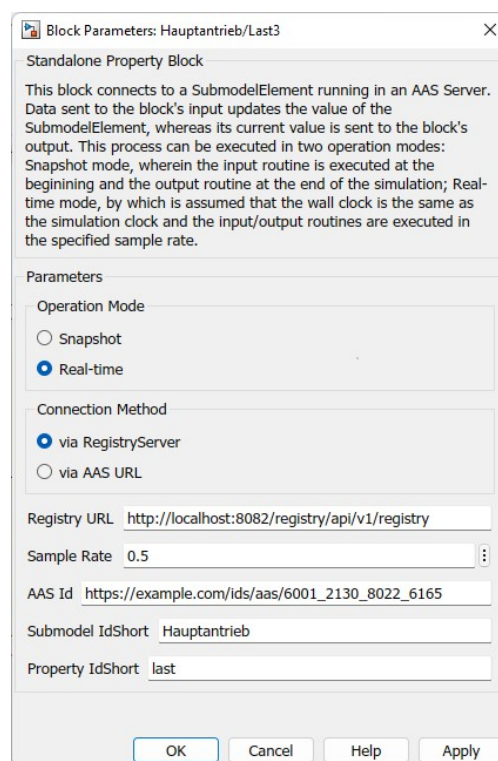


Abbildung 3.10: Dialogfenster des PropertySACconnector-Block

Der Benutzer kann über das Fenster den gewünschten Betriebsmodus auswählen, wobei bei Wählen des Echtzeitmodus das Feld zur Einstellung der Abtastrate sichtbar wird. Es steht außerdem zur Verfügung die Möglichkeit, auszuwählen, ob es das *AAS* über den *RegistryServer* oder direkt über seine Adresse abgerufen werden soll.

Kapitel 4

Validierung

In diesem Kapitel liegt der Schwerpunkt an der Vorstellung von dem Beispielszenario, genutzt, um die richtige Umsetzung der sämtlichen Anforderungen zu veranschaulichen. Bei dem ersten Teilkapitel wird dessen Aufbau besprochen, während bei dem zweiten und letzten es einen Überblick bezüglich der Erwartungen und der aus dem Beispiel ausgegebenen Ergebnisse gewonnen werden soll.

4.1 Beispielszenario

Für das Testen des Systems wurde ein Beispielszenario entwickelt, bei dem alle Fähigkeiten der in der Arbeit entwickelten Bibliothek beobachtet werden können. Dafür wurde ein vereinfachtes Beispiel vom *Condition-Monitoring* realisiert, bei dem die Funktionsweise eines Temperatursensors und des Hauptantriebssystems einer Pressmaschine im Simulink nachgebildet und deren digitale Zwillinge in einem BaSyx-Server gespeichert werden. Die Temperaturdaten werden letztendlich in einem in Simulink realisierten Dashboard dargestellt.

4.1.1 Setup vom BaSyx/Docker

Beide für das Beispiel verwendeten Registry- und AAS-Server nehmen die verfügbaren *docker containers* von *BaSyx* im Anspruch. Bei Nutzung von *docker-compose* wird eine schnelle Bereitstellung von beiden notwendigen *containers* möglich. Im Codeauszug 7 wird der Aufbau der im Beispiel eingesetzten Konfigurationsdatei für *docker-compose*.

Beim Anfahren lädt der AAS-*container* das im Abb. 4.1 dargestellte AAS. Dieses enthält zwei *Submodels* für vereinfachte Abbildung von beiden Hauptantrieb und Temperatursensor. Diese weisen wiederum je ein *SubmodelElement* auf, die entsprechend zur Darstellung der aktuellen Last am Antriebe und des aktuellen erfassten Temperaturdatenpunktes dienen.

4.1.2 Aufbau

Das Simulink-Modell (Abb. 4.2) besteht aus 4 Teilsystemen:

1. Last: aktualisiert den Wert vom *Hauptantrieb/last SubmodelElement* mit dem Wert einer Schiebekomponente.
2. Hauptantrieb: bildet die Funktionsweise des Hauptantriebes nach und gibt den Augenblickswert der Temperatur am Kontakt zwischen Antriebe und Sensor.
3. Temperatursensor: bildet die Funktionsweise des Temperatursensors und schickt den abgetasteten Wert an *TempSensor/temperatur SubmodelElement*.

```
# docker/docker-compose.yml
version: '2.1'
services:
  registry:
    image: eclipsebasyx/aas-registry:1.1.0
    container_name: registry
    volumes:
      - ./conf/reg-config:/usr/share/config:z
    ports:
      - 8082:4000

  aas:
    image: eclipsebasyx/aas-server:1.1.0
    container_name: aas
    volumes:
      - ./conf/aas-config:/usr/share/config:z
    ports:
      - 4001:4001
```

Listing 7: Auszug aus der Datei docker/docker-compose.yml

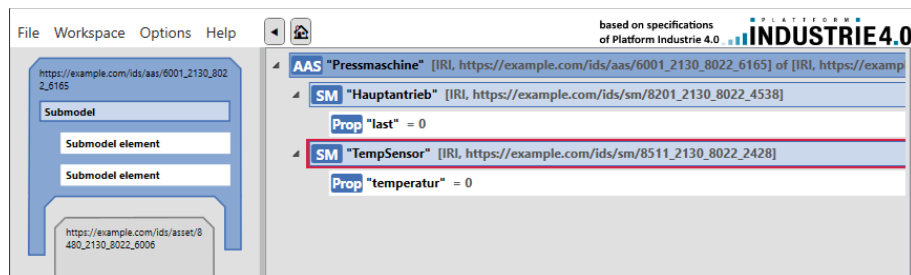


Abbildung 4.1: Bildschirmaufnahme der Verwaltungsschale auf AASX Package Explorer

4. Dashboard: erhält und verarbeitet die Daten aus dem *TempSensor/temperatur SubmodelElement* und ermöglicht *Monitoring* über einen Graf

Beim Einstellen des Schiebers im Last-Teilsystem ändert sich die Temperatur vom Hauptantriebe. Dieser Effekt wird sich auf das Temperatursensor-Teilsystem fortgepflanzt, wobei den Temperaturdaten erfassen und an den *TempSensor/temperatur SubmodelElement* übertragen werden. Anschließend werden die Daten im Dashboard-Teilsystem über einen zusätzlichen *PropertySAConnectorBlock* angefragt, in einem späteren Block verarbeitet und im einen Graf dargestellt. Der Graf ist in 3 Bereichen aufgeteilt: in grünen Bereich wird die Temperaturen vom normalen Betrieb dargestellt, wobei in gelben und roten sind entsprechend der Warnzustand und der Notzustand erkennbar.

4.2 Ausführung und Prüfen der Anforderungen

Bei dem Test hat es sich herausgestellt, dass das System in der Lage war, alle Anforderungen angemessen zu realisieren. Das Beispiel wies vielfache Instanzen des Blockes *PropertySAConnectorBlock* auf, und beide Lese- und Aktualisierungsanfrage erfordert hat. Darüber hinaus wurden alle für seine Funktionsweise relevanten Parameter direkt im Simulink eingestellt.

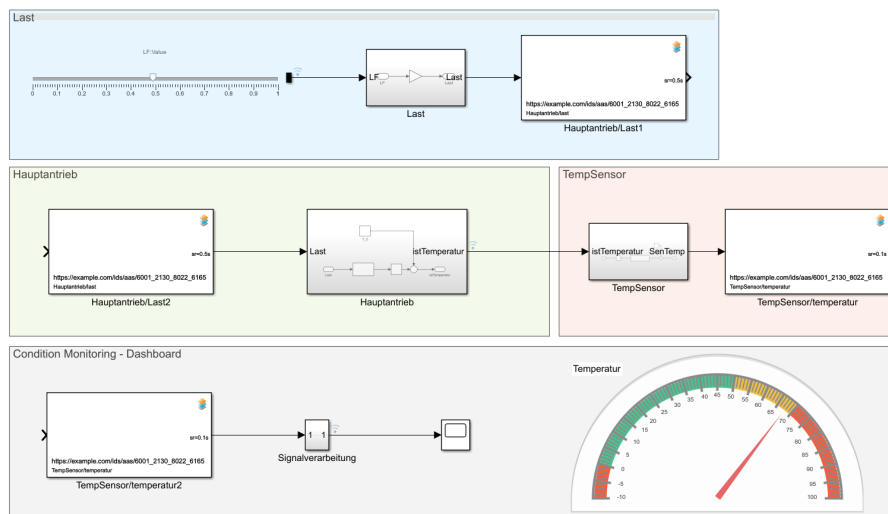


Abbildung 4.2: Simulink-Modell des Beispielszenarios

Im Abbild 4.3 werden die Verläufe von dem Augenblickswert und dem vom Sensor abgetasteten Wert der Temperatur an der Kontaktstelle bei variiertem Lastfaktor angezeigt.

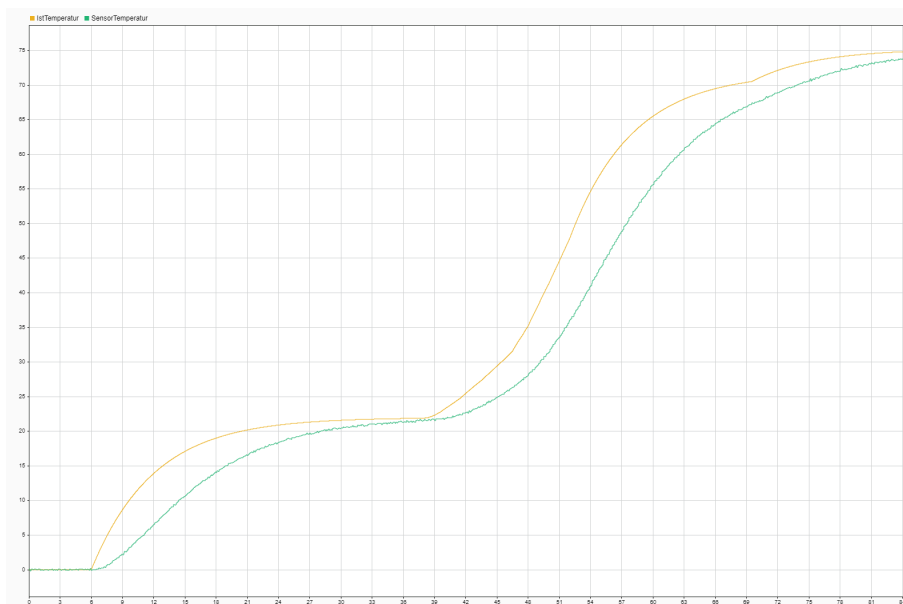


Abbildung 4.3: Verläufe von dem Augenblickswert und dem vom Sensor abgetasteten Wert der Temperatur an der Kontaktstelle

Die Simulation wurde im Echtzeitmodus betrieben, bei dem die Simulationszeit der Wanduhrzeit entsprach. Dieser Modus wurde vorgezogen, weil damit sich die Zusammenwirkung BaSyx und Simulink besser vorstellen lässt. Der Abbild-Modus ist prinzipiell sinnvoll, wenn die Simulation asynchron bezüglich der Hardware-, bzw. Software-in-the-loop abläuft.

Es lohnt sich erwähnen, dass so weit Anforderungen bezüglich der höchsten möglichen Abtastrate nicht betrachtet worden sind. Jedoch wurden mehrere Blöcke mit Abtastraten bis 1KHz im Beispiel erfolgreich verwendet.

Kapitel 5

Zusammenfassung und Ausblick

5.1 Fazit

5.2 Ausblick

Abbildungsverzeichnis

2.1	Struktur eines Simulink-Blockes [8]	4
2.2	Stufen des Industrie 4.0-Entwicklungspfad (FIR e. V., RWTH Aachen University [10]	6
2.3	Beispiel einer einfachen BaSys-Architektur [4]	6
2.4	Abbild eines AAS [4]	7
2.5	VAB-Beispiel mit verbundenen Entitäten [4]	8
2.6	Diagramm einer Eclipse BaSysx-Systemstruktur [4]	9
3.1	Funktionsbausteine jeder Teilkomponente	13
3.2	Darstellung des Kommunikationsflusses vom A1, A2 und A4	14
3.3	Klassendiagramm	15
3.4	Sequenzdiagramm für A1	19
3.5	Sequenzdiagramm für A2	19
3.6	Sequenzdiagramm für A3	22
3.7	Beispiel einer Fehlermeldung	22
3.8	Simulink-Modell mit mehreren Blöcken	23
3.9	Instanz eines PropertySACconnector-Block	24
3.10	Dialogfenster des PropertySACconnector-Block	24
4.1	Bildschirmaufnahme der Verwaltungsschale auf AASX Package Explorer	28
4.2	Simulink-Modell des Beispielszenarios	29
4.3	Verläufe von dem Augenblickswert und dem vom Sensor abgetasteten Wert der Temperatur an der Kontaktstelle	29

Tabellenverzeichnis

1.1	Anforderungstabelle	2
-----	-------------------------------	---

Literatur

- [1] Enlyft. „Simulink market share.“ (), Adresse: <https://enlyft.com/tech/products/simulink> (besucht am 31.03.2022).
- [2] The MathWorks, Inc., *Simulink Getting Started Guide*.
- [3] Fraunhofer-Institut für Experimentelles Software Engineering IESE und Thomas Kuhn, *BaSys 4.0 Flyer*. Adresse: https://www.basys40.de/wp-content/uploads/2019/12/Flyer_BaSys40_deutsch.pdf (besucht am 31.03.2022).
- [4] Eclipse Foundation, Inc. „BaSyx - Eclipsepedia.“ (), Adresse: <https://wiki.eclipse.org/BaSyx> (besucht am 31.03.2022).
- [5] M. Zimmer, *Durchgängiger simulationsprozess zur effizienzsteigerung und reifegraderhöhung von konzeptbewertungen in der frühen phase der produktentstehung*. Wiesbaden: Springer Fachmedien Wiesbaden, 2015, ISBN: 978-3-658-11499-2 978-3-658-11500-5. DOI: 10.1007/978-3-658-11500-5. Adresse: <http://link.springer.com/10.1007/978-3-658-11500-5> (besucht am 02.04.2022).
- [6] A. Bosl, *Einführung in MATLAB/Simulink*, 3., vollständig überarbeitete Auflage. Hanser, 2020, ISBN: 978-3-446-46546-6.
- [7] M. Eigner, D. Roubanov und R. Zafirov, Hrsg., *Modellbasierte virtuelle produktentwicklung*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2014, ISBN: 978-3-662-43815-2 978-3-662-43816-9. DOI: 10.1007/978-3-662-43816-9. Adresse: <http://link.springer.com/10.1007/978-3-662-43816-9> (besucht am 02.04.2022).
- [8] The MathWorks, Inc. „What Is an S-function? - MATLAB & Simulink - MathWorks Deutschland.“ (), Adresse: <https://de.mathworks.com/help/simulink/sfg/what-is-an-s-function.html> (besucht am 03.04.2022).
- [9] T. Kuhn, F. Schnicke und P. Oliveira Antonino, „Service-Based Architectures in Production Systems: Challenges, Solutions Amp; Experiences,“ in *2020 ITU Kaleidoscope: Industry-Driven Digital Transformation (ITU K)*, Dez. 2020, S. 1–7. DOI: 10.23919/ITUK50268.2020.9303207.
- [10] G. Schuh, R. Anderl, R. Dumitrescu und A. Krüger, „Industrie 4.0 maturity index,“ *acatech*, S. 64,
- [11] P. Chourtsidis, „Scoping the industry 4.0 reconfigurability,“ Technische Universität Kaiserslautern, 6. Nov. 2020. Adresse: <https://kluedo.ub.uni-kl.de/frontdoor/index/index/docId/6488> (besucht am 24.03.2022).
- [12] Plattform Industrie 4.0, „Diskussionspapier Verwaltungsschale in Der Praxis: Verwaltungsschale in Der Praxis.“ Adresse: https://www.plattform-i40.de/IP/Redaktion/DE/Downloads/Publikation/2019-verwaltungsschale-in-der-praxis.pdf?__blob=publicationFile&v=7 (besucht am 01.04.2022).
- [13] S. Reinheimer, Hrsg., *Industrie 4.0: herausforderungen, konzepte und praxisbeispiele* (Edition HMD). Wiesbaden: Springer Fachmedien Wiesbaden, 2017, ISBN: 978-3-658-18164-2 978-3-658-18165-9. DOI: 10.1007/978-3-658-18165-9. Adresse: <http://link.springer.com/10.1007/978-3-658-18165-9> (besucht am 01.04.2022).
- [14] H. Balzert und P. Liggesmeyer, *Lehrbuch der softwaretechnik: entwurf, implementierung, installation und betrieb* (Lehrbücher der Informatik), 3. Aufl. Heidelberg: Spektrum Akademischer Verl, 2011, ISBN: 978-3-8274-2246-0.

Titel der Arbeit:

Ermöglichung des Hardware- und Software-in-the-Loop durch
die Einbindung von BaSyx und MATLAB/Simulink

Art der Arbeit und Datum:

Projektarbeit, Juni 2022

Betreuung:

Dipl.-Ing. Paaranan Sivasothy
Prof. Dr.-Ing. Jörg Seewig

Student:

Name: Mateus Molina
E-mail: molina@rhrk.uni-kl.de
Matrikelnr.: 420495
Semester: 3

Erklärung betreffend Plagiaten:

Ich erkläre mit meiner Unterschrift die vorliegende Arbeit selbständig verfasst und die im betroffenen Fachgebiet üblichen Zitiervorschriften eingehalten zu haben.

Kaiserslautern, 13. 8. 2022: _____